

Introdução ao Python

<https://github.com/laurogripa/python-udesc>



UDESC
UNIVERSIDADE
DO ESTADO DE
SANTA CATARINA

Lauro Gripa Neto

Prof. Dr. Marcelo da Silva Hounsell

Computação Gráfica Avançada

Mestrado em Computação Aplicada

15/06/2026

- Graduado em **Tecnologia em Análise e Desenvolvimento de Sistemas** pela Universidade do Estado de Santa Catarina, em 2016.
- Começou a programar por hobby aos **13 anos**, com mIRC Script.
- Iniciou profissionalmente aos **22 anos**, com PHP e JavaScript.
- **2,5 anos** no desenvolvimento de ERPs.
- **5 anos** em consultoria e *outsourcing* especializado, trabalhando com diversas linguagens.
- Experiência principalmente com **Ruby on Rails**.
- Breve experiência profissional com **Python**, em 2019.
- Nos últimos **5 anos**, atua como freelancer em Web3/Blockchain com TypeScript, React e Rust.
- Especializou-se em **Web, Desenvolvimento Ágil e Sistemas Descentralizados (Blockchain)**.

- História
- Filosofia
- Características
- Vantagens
- Limitações
- Usos comuns
- Exemplos
- Exercícios

- **30 minutos de mentoria** para cada equipe.
- Todas as equipes terão o mesmo direito.
- Basta marcar um horário pelo <https://calendly.com/lauro-gripa/30min>.
- Confirmação por e-mail.

História do Python

- Criado por **Guido van Rossum**; primeira versão pública em **20 de fevereiro de 1991**.
- Nome inspirado no grupo de comédia **Monty Python**.



Guido van Rossum
“Benevolent dictator for life” (1995–2018)

- Desenvolver uma linguagem **acessível**, com sintaxe clara e adequada ao trabalho cotidiano.
- Manter a **simplicidade e legibilidade** da linguagem ABC.
- Superar a dificuldade de **extensão e integração** da ABC.
- Automatizar tarefas no sistema operacional distribuído **Amoeba**.
- Oferecer uma alternativa mais prática que **C** para scripts.
- Permitir integração fácil com módulos e bibliotecas escritos em **C**.

Filosofia da linguagem: o Zen do Python

Bonito é melhor que feio.	Erros nunca devem passar silenciosamente.
Explícito é melhor que implícito.	A menos que sejam explicitamente silenciados.
Simple é melhor que complexo.	Diante da ambiguidade, recuse a tentação de adivinhar.
Complexo é melhor que complicado.	Deve haver uma maneira óbvia de fazer algo.
Plano é melhor que aninhado.	Mesmo que ela não seja óbvia à primeira vista.
Esparso é melhor que denso.	Agora é melhor que nunca.
Legibilidade conta.	Nunca pode ser melhor que agora mesmo.
Casos especiais não quebram as regras.	Se é difícil explicar, a ideia é ruim.
A praticidade vence a pureza.	Se é fácil explicar, pode ser uma boa ideia.
	<i>Namespaces</i> são uma ótima ideia: vamos usá-los mais!

- Execução **interpretada**, normalmente a partir de bytecode.
- Suporte a múltiplos **paradigmas de programação**.
- Blocos de código definidos por **indentação significativa**.
- Tipagem **dinâmica e forte**.
- **Gerenciamento automático de memória**.
- Ampla biblioteca padrão e ecossistema **extensível**.

Python é multiparadigma

Paradigma	Na prática
Programação imperativa	Instruções alteram o estado do programa passo a passo.
Programação procedural	Scripts organizam tarefas em funções reutilizáveis.
Programação orientada a objetos	Aplicações maiores usam classes e objetos.
Programação funcional	Funções como <code>map()</code> e <code>filter()</code> transformam dados.

Os paradigmas podem coexistir no mesmo projeto.

Compilado vs. interpretado

Critério	Compilado	Interpretado
Tradução	Antes da execução	Durante a execução, com auxílio do interpretador
Resultado	Código de máquina ou outra representação	Execução mediada por outro programa
Característica	Geralmente oferece maior desempenho	Favorece portabilidade e experimentação
Exemplo típico	C	Python com CPython

A distinção depende da implementação: uma linguagem pode combinar compilação e interpretação.

Python

```
print("Hello, World!")
```

C

```
#include <stdio.h>

int main(void) {
    printf("Hello, World!\n");
    return 0;
}
```

- O código-fonte é executado com a ajuda de um **interpretador**.
- Em CPython, o código é transformado em **bytecode** antes da execução.
- O bytecode pode ser armazenado em arquivos `.pyc`.
- **CPython** é a implementação principal e mais utilizada da linguagem.
- A execução difere de linguagens compiladas diretamente para código de máquina, como C.

Java também usa uma representação intermediária; JavaScript normalmente é executado por mecanismos com interpretação e compilação JIT.

Dinâmica

- O tipo está associado ao **valor**, não à variável.
- Uma variável pode receber valores de tipos diferentes.
- Menos declarações são necessárias no código inicial.

Forte

- Python evita conversões implícitas incompatíveis.
- Operações entre tipos inadequados geram erro.
- Conversões devem ser feitas de forma explícita.

Vantagens

- Menos código inicial
- Maior flexibilidade
- Experimentação rápida
- APIs simples de usar

Problemas comuns

- Erros descobertos em execução
- Manutenção mais difícil em projetos grandes
- Refatorações mais arriscadas
- Contratos pouco claros entre funções

Type hints documentam os tipos esperados e permitem análise estática, sem remover a tipagem dinâmica da linguagem.



- O interpretador adiciona custo à execução.
- A tipagem dinâmica exige verificações em tempo de execução.
- Em CPython, o **GIL** limita a execução simultânea de bytecode por múltiplas threads.
- Python pode ser mais lento que linguagens compiladas em tarefas intensivas de CPU.
- Em automação, integração e prototipagem, o tempo de desenvolvimento costuma ser mais importante.

Bibliotecas otimizadas

- NumPy
- Pandas
- OpenCV
- Implementações nativas em C/C++

Outras alternativas

- PyPy
- Cython
- `multiprocessing`
- Partes críticas em C, C++ ou Rust

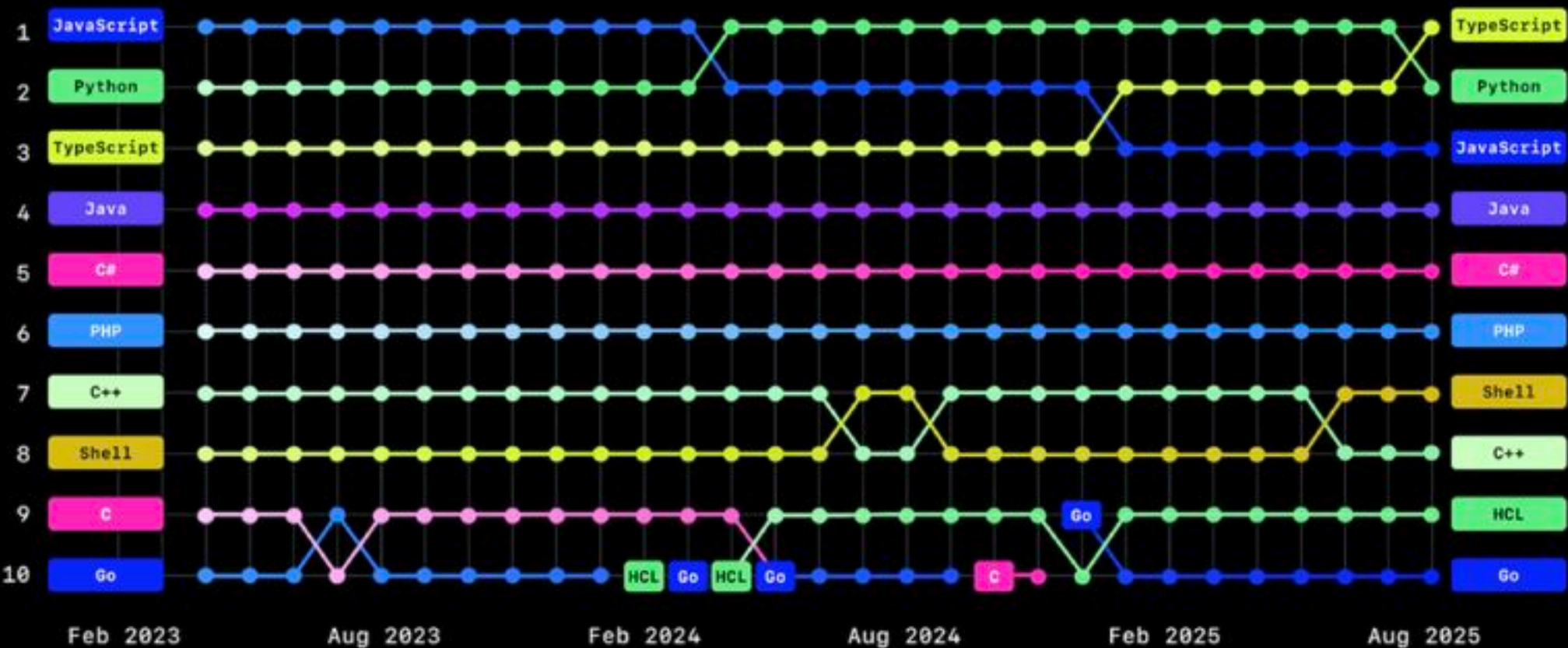
Primeiro identifique o gargalo; depois escolha a estratégia adequada.

Por que Python é bom para prototipagem?

- Sintaxe simples e legível
- Pouco código para produzir resultados úteis
- Facilidade para testar ideias rapidamente
- REPL e notebooks para experimentação
- Grande quantidade de bibliotecas
- Boa curva de aprendizado
- Aplicação em scripts, automações e MVPs

- Ciência de dados
- Inteligência artificial e *machine learning*
- Automação de tarefas
- Desenvolvimento web
- APIs e backends
- Educação e ensino de programação
- Jogos simples e protótipos
- Testes e ferramentas internas
- Segurança e redes
- DevOps

Top 10 programming languages on GitHub 2023-2025



Estrutura

- Indentação obrigatória
- Comentários com `#`
- Blocos sem chaves
- Código organizado em arquivos `.py`

Variáveis e tipos

- `int`
- `float`
- `str`
- `bool`
- Operadores aritméticos e lógicos

- `print()` exibe valores na saída padrão.
- `input()` lê uma linha digitada pelo usuário.
- O resultado de `input()` é sempre uma string.
- Funções como `int()`, `float()` e `str()` convertem valores.

Entradas inválidas precisam ser tratadas para evitar erros em tempo de execução.

Decisão

- `if`
- `elif`
- `else`

Expressões

- Operadores relacionais
- `and`, `or` e `not`
- Valores verdadeiros e falsos
- Condições combinadas

`while`

- Repete enquanto uma condição for verdadeira.
- Útil quando a quantidade de repetições não é conhecida.
- Exige cuidado com loops infinitos.

`for`

- Percorre itens de uma sequência.
- `range()` produz sequências numéricas.
- `break` interrompe o loop.
- `continue` avança para a próxima repetição.

- **Listas:** ordenadas e mutáveis
- **Tuplas:** ordenadas e imutáveis
- **Dicionários:** pares de chave e valor
- **Conjuntos:** valores únicos sem ordem garantida
- Strings também são sequências.
- Indexação acessa uma posição.
- *Slicing* seleciona partes de uma sequência.
- Loops permitem percorrer todos os elementos.

- Funções são definidas com `def` .
- Parâmetros recebem dados de entrada.
- `return` devolve um resultado.
- Variáveis criadas dentro da função possuem escopo local.
- Funções reduzem repetição e organizam responsabilidades.

Uma boa função realiza uma tarefa clara e possui entradas e saídas compreensíveis.

Reutilização

- `import` carrega módulos.
- A biblioteca padrão oferece recursos prontos.
- Pacotes agrupam módulos relacionados.

Organização

- Separar responsabilidades em arquivos
- Evitar arquivos excessivamente grandes
- Reutilizar funções e classes
- Tornar dependências explícitas

- `try` envolve uma operação que pode falhar.
- `except` trata uma exceção esperada.
- Mensagens claras ajudam o usuário a corrigir a entrada.
- Exceções específicas evitam esconder erros inesperados.

Erros comuns de iniciantes incluem indentação incorreta, nomes inexistentes, conversões inválidas e acesso a índices fora da coleção.

- Criar uma janela
- Entender a estrutura básica de um jogo
- Processar eventos
- Atualizar o estado
- Desenhar na tela
- Repetir essas etapas no loop principal

Entrada e atualização

- Capturar teclado
- Processar eventos
- Mover um objeto
- Detectar colisão simples

Desenho e controle

- Limpar a tela
- Desenhar formas
- Atualizar a janela
- Controlar FPS
- Encerrar corretamente

- Python é uma linguagem acessível, produtiva e multiparadigma.
- A tipagem dinâmica acelera o desenvolvimento, mas exige disciplina.
- O ecossistema torna Python útil em diversas áreas.
- A linguagem é uma boa escolha para automação, ensino e prototipagem.
- Restrições de desempenho podem exigir bibliotecas ou tecnologias complementares.

Estudo

- Praticar sintaxe e coleções
- Criar funções pequenas
- Organizar código em módulos
- Aprender a ler mensagens de erro

Desafio com Pygame

- Mover um objeto com o teclado
- Impedir que ele saia da janela
- Adicionar um obstáculo
- Detectar e indicar uma colisão

- PYTHON SOFTWARE FOUNDATION. **The Python Language Reference**. Python 3.14.6 Documentation, 2026. Disponível em: <https://docs.python.org/3/reference/>. Acesso em: 15 jun. 2026.
- LEARNPYTHON.ORG. **Learn Python: Free Interactive Python Tutorial**. [s.d.]. Disponível em: <https://www.learnpython.org/>. Acesso em: 15 jun. 2026.
- FREE SOFTWARE FOUNDATION. **GNU C Language Manual**. [s.d.]. Disponível em: https://www.gnu.org/software/c-intro-and-ref/manual/html_node/index.html. Acesso em: 15 jun. 2026.
- STACK OVERFLOW. **2025 Developer Survey: Technology**. 2025. Disponível em: <https://survey.stackoverflow.co/2025/technology>. Acesso em: 15 jun. 2026.
- HEER, N. **Speed comparison of programming languages**. GitHub, [s.d.]. Disponível em: <https://github.com/niklas-heer/speed-comparison>. Acesso em: 15 jun. 2026.
- HARRIS, C. R. et al. Array programming with NumPy. **Nature**, v. 585, p. 357–362, 2020. DOI: <https://doi.org/10.1038/s41586-020-2649-2>.